3주차 4일차: Toolkit으로 MVVM 기본 구조 완성 — 120분

1. 학습 목표 — 5분

- 반복되는 속성 알림을 ObservableObject와 SetProperty로 줄입니다.
- 이벤트 방식과 MVVM 방식의 차이를 같은 화면에서 정리합니다.
- 다음 주 API 결과를 받을 작은 MVVM 프로그램을 준비합니다.

시작 상태는 day11입니다.

2. 핵심 개념 설명 — 15분

지금 MainViewModel과 Post에는 PropertyChanged 이벤트를 선언하고 호출하는 코드가 반복됩니다. ObservableObject는 이 알림 기능을 제공하는 Toolkit의 기반 클래스입니다. SetProperty는 값이 달라졌는지 확인하고, 필드에 저장하고, 속성 변경을 알립니다. 공식 ObservableObject 안내

set => SetProperty(ref title, value)는 짧게 쓴 속성 설정 코드입니다. ref는 SetProperty가 실제 title 필드를 바꿀 수 있게 전달하는 C# 표기입니다. Input은 값이 달라진 경우 명령 상태도 재확인해야 하므로 if 문으로 SetProperty의 반환값을 사용합니다.

1주차 방식	오늘 방식
창의 Click 처리기	ViewModel의 RunCommand
InputTextBox.Text 읽기	Input 속성 읽기
ResultTextBlock.Text 쓰기	Result 변경과 속성 알림
화면 컨트롤을 직접 조작	데이터·명령을 Binding으로 연결

이번 완료본에서는 2주차의 "첫 행 수정 + 행 추가" 관찰용 동작을 정리합니다. 실행할 때 기존 목록을 지우고 입력 텍스트 한 건을 결과 표에 넣습니다. 다음 주에는 이 자리에 API 결과가 들어갑니다. 버튼을 추가하지 않습니다.

소스 생성기 특성, DI, 메시징, 서비스 계층은 사용하지 않습니다. 현재 Code-behind에 InitializeComponent와 DataContext 설정이 남아 있어도 역할에 맞는 코드입니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
private string input = "안녕하세요, WPF";
public string Input
{
    get => input;
    set
    {
        if (SetProperty(ref input, value))
        {
            RunCommand.NotifyCanExecuteChanged();
        }
    }
}
```

Result와 Status도 SetProperty를 사용합니다. 직접 선언했던 event와 Notify 메서드는 제거합니다. 실행의 핵심은 다음과 같습니다.

```
    Posts.Clear();
    Posts.Add(new Post { Id = 1, Title = text });
```

창의 C# 파일은 그대로 두고, ViewModel과 Post의 알림 코드를 간단하게 바꿉니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
Post.cs	day12의 Post.cs
MainViewModel.cs	day12의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 75분

코드 전환 — 25분

1. 창을 닫고 MainViewModel.cs와 Post.cs를 오늘의 전체 코드로 교체합니다.
2. 두 클래스가 ObservableObject를 상속하는지 확인합니다.
3. Result·Status·Title의 set에 SetProperty가 있는지 봅니다. MainViewModel에서 직접 선언한 PropertyChanged와 Notify가 없어졌는지도 확인합니다.
4. 솔루션을 빌드합니다. Toolkit 관련 형식을 찾지 못하면 8.4.2 패키지 복원 상태를 확인합니다.

같은 화면으로 동작 확인 — 20분

1. F5로 실행합니다. 이번 완료본은 결과 목록이 비어 있는 상태로 시작합니다.
2. "MVVM 테스트"를 입력하고 실행하면 번호 1의 한 행이 나타납니다.
3. 다른 입력을 실행하면 이전 행 대신 새 텍스트 한 건이 나타납니다.
4. 입력을 지우면 버튼이 비활성화되고, 다시 넣으면 활성화되는지 봅니다.

데이터가 흐르는 순서 관찰 — 20분

1. Input의 set에 중단점을 걸고 입력합니다. 화면값이 ViewModel로 들어오는 것을 확인합니다.
2. 중단점을 옮겨 Run의 Result 대입문에서 멈춥니다. F10으로 값을 바꾸면 속성 알림을 통해 화면이 갱신됩니다.
3. MainWindow.xaml에서 Input·RunCommand·Posts·Result·Status 바인딩을 차례로 찾습니다.
4. 중단점을 모두 제거하고 실행을 종료합니다.

다음 주를 위한 저장 — 10분

1. 다시 실행해 입력·명령·결과 표의 연결을 확인합니다.
2. 파일을 역할로 구분해 봅니다. Post는 데이터, XAML은 화면, MainViewModel은 화면에 제공할 상태와 동작입니다.
3. 저장하고 종료합니다. 새 프로젝트는 만들지 않습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 11로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 12로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- ObservableObject와 SetProperty는 지난주 배운 변경 알림의 반복 구현을 대신합니다.
- Toolkit을 써도 DataContext·Binding·속성 알림의 원리는 같습니다.
- ViewModel에는 화면 컨트롤을 직접 참조하는 코드가 없습니다.
- 구조를 복잡하게 확장하지 않고, 다음 주에는 Run만 비동기 요청으로 바꿉니다.

완료본: day12. 다음: 4주차 1일차.